

LIM-ARC: gent MoE

gent OS

<https://slim.nexa-lang.com/>

0 About us

➤ gent OS (T2026105589911358)



➤ Proj59

➤ <https://slim.nexa-lang.com/>



Catalogue

- ❖ 1 Background & Motivation
- ❖ 2 Project Introduction
- ❖ 3 SLIM-ARC Core Design
- ❖ 4 Projrct Implementation
- ❖ 5 Testing & Evaluation
- ❖ 6 Summary & Outlooking

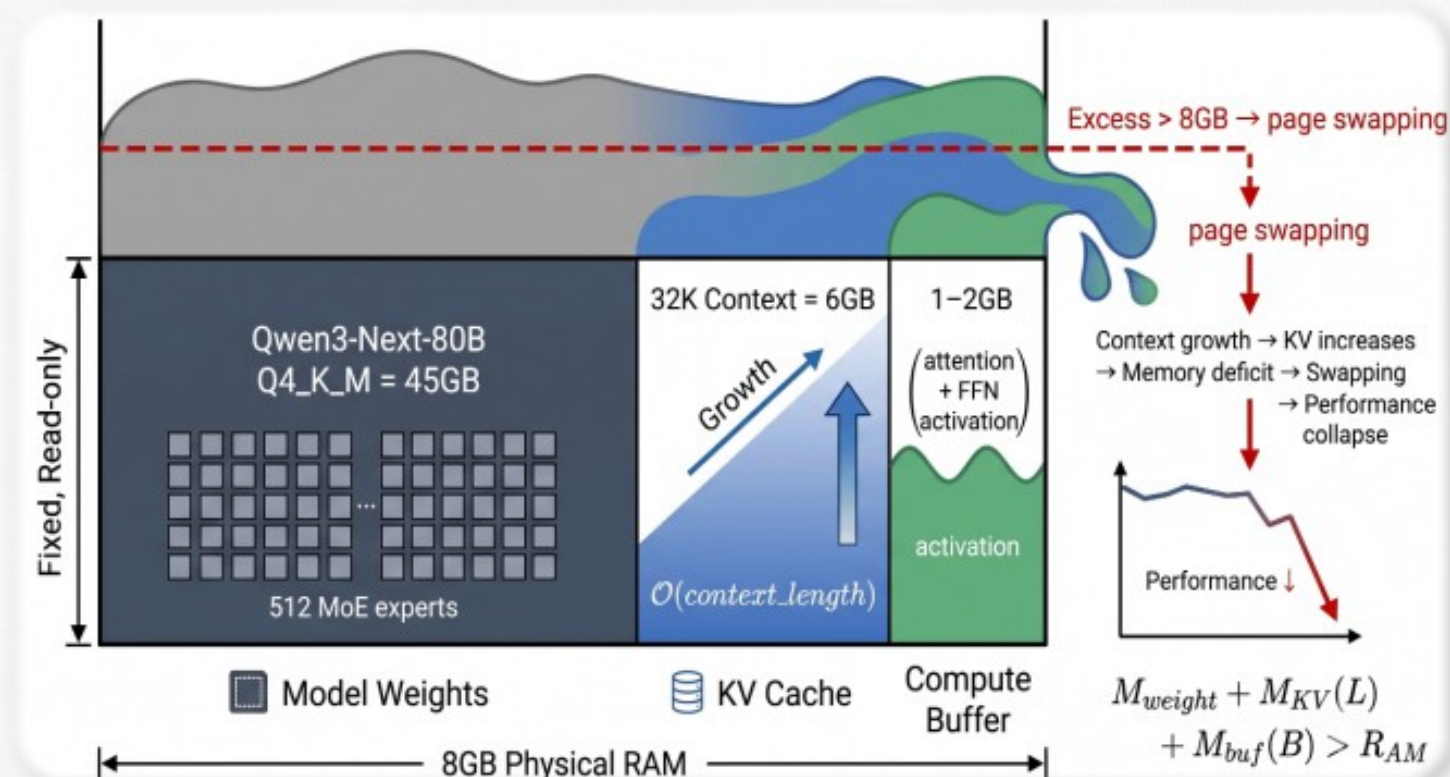
1

gent



80B Qwen3-
Next-80B 45GB
140GB
8 16GB

KV Cache



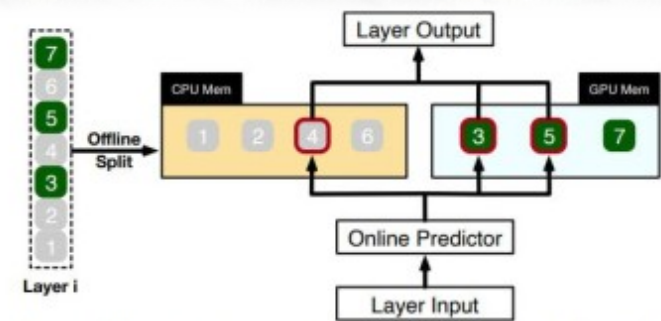
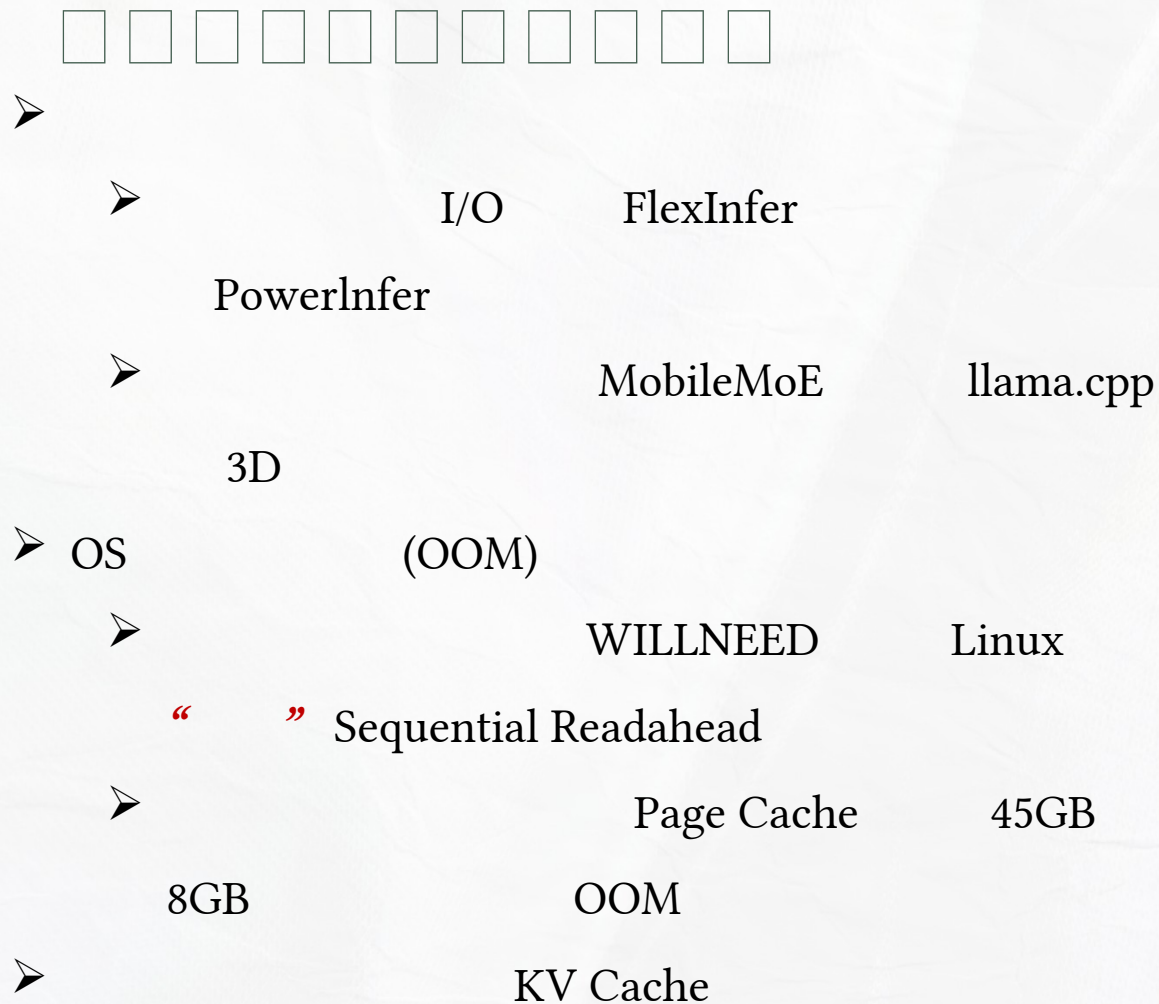


Figure 8: An illustrative example shows how PowerInfer calculates different neurons for one LLM layer.

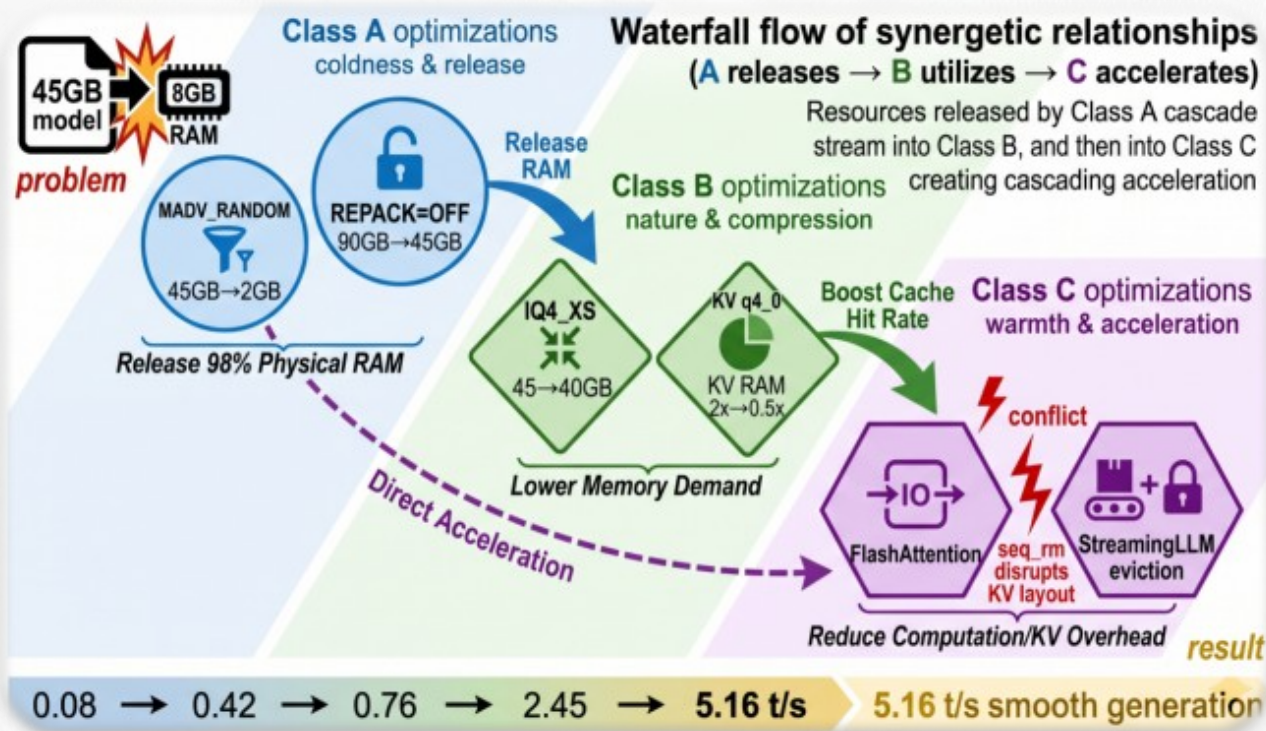
4.2 Single Layer Example

Figure 8 illustrates how PowerInfer coordinates GPU and CPU in processing a layer’s neurons. It classifies neurons based on offline data, assigning hot-activated ones (e.g., indices 3, 5, 7) to GPU memory and others to CPU memory. Upon receiving an input, a predictor identifies which neurons in the current layer are likely to be activated. For instance, it predicts activation for neurons 3, 4, and 5. It is crucial to note that hot-activated neurons, identified through offline statistical analysis, may not consistently match the runtime activation behaviors. For example, neuron 7, though labeled as hot-activated, is forecasted to be inactive in this case.

Both CPU and GPU then process predicted active neurons, ignoring inactive ones. The GPU computes neurons 3 and 5, while the CPU handles neuron 4. Once neuron 4’s computation is complete, its output is sent to the GPU for result integration.

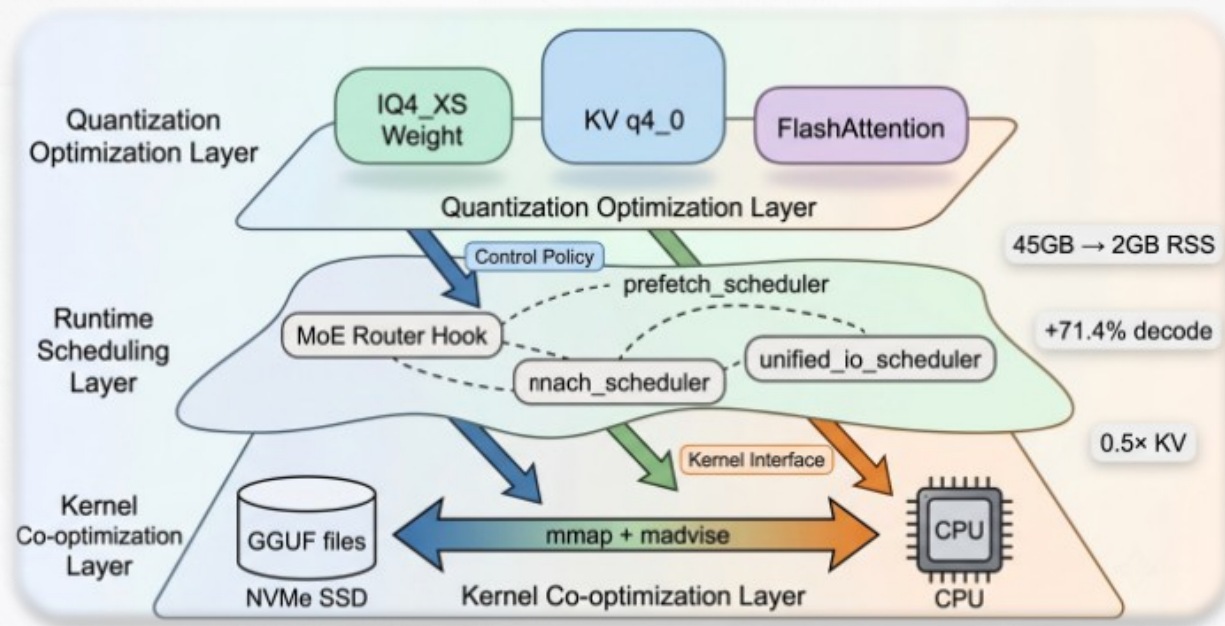
2

- OS
- (Cascading Synergy)
- (Class A) OS
- (Class B) RAM
- (Class C) I/O
- “ → → ”
- MoE



3

SLIM- RC



➤ OS

➤

➤

➤

➤ ***MMAP*** 45GB➤ ***POSIX***

➤

MoE

KV

I/O

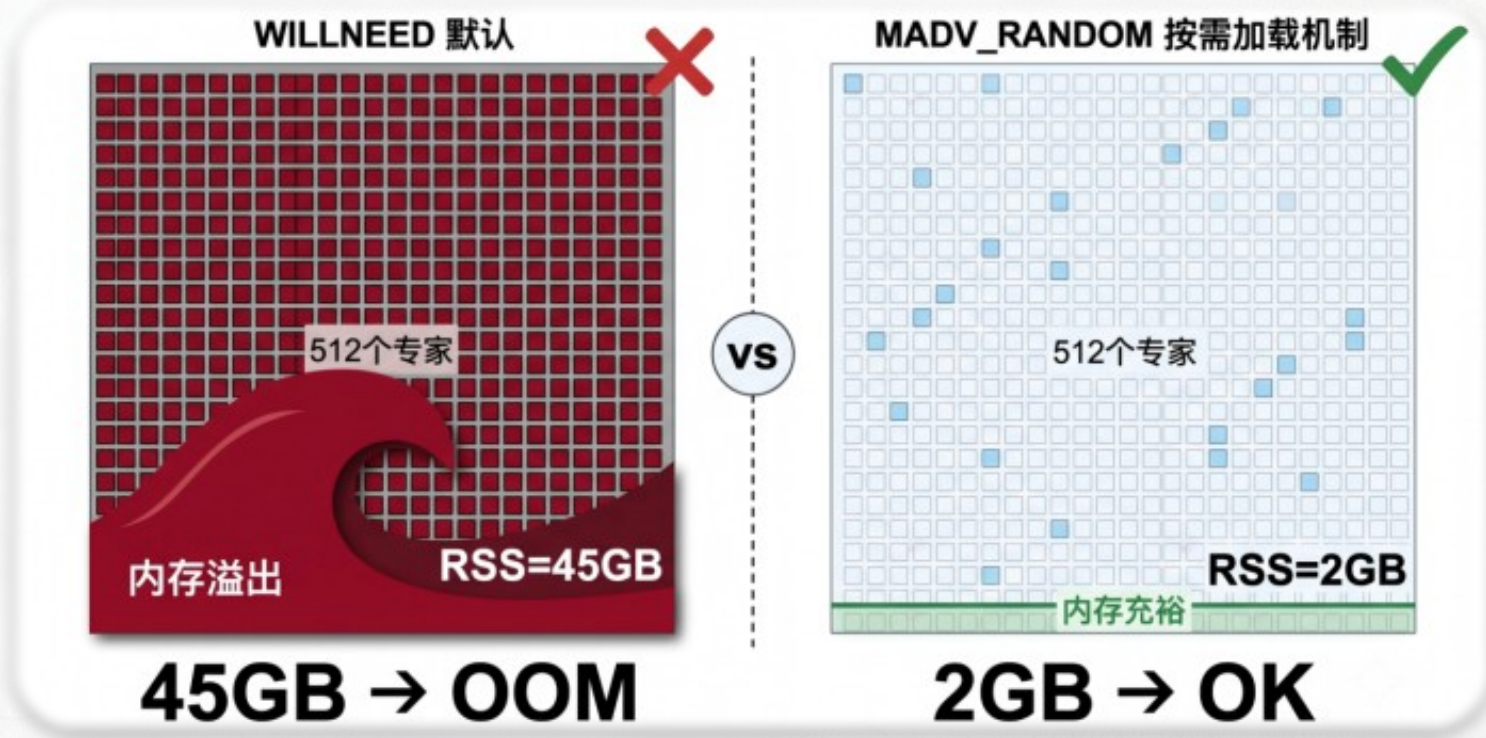
➤

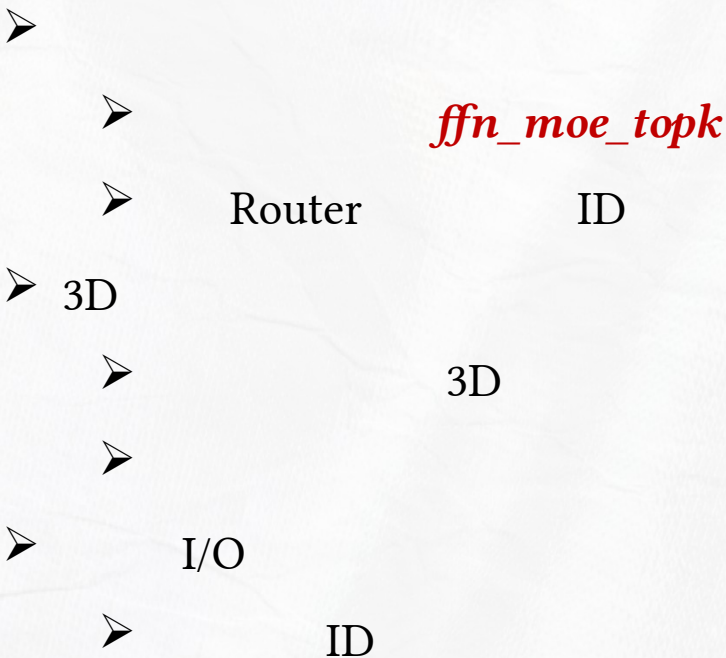
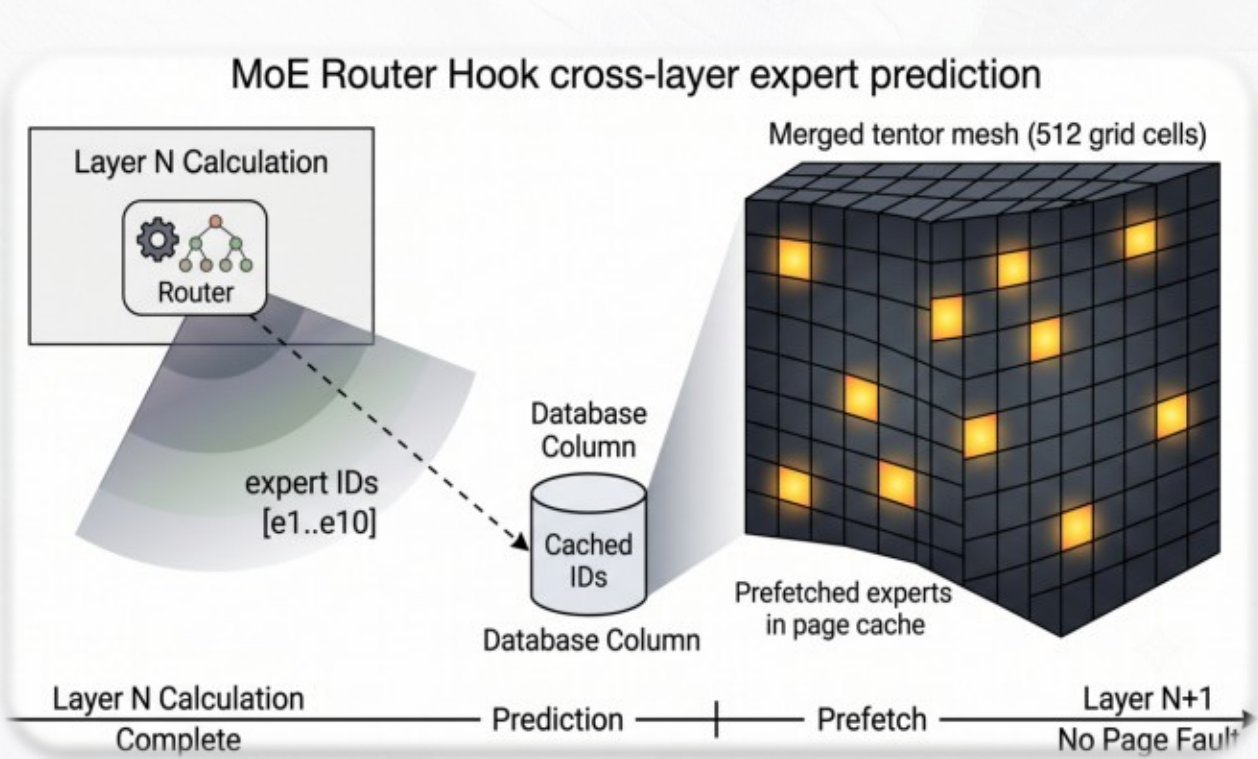
IQ4_XS

FlashAttention

MADV_RANDOM

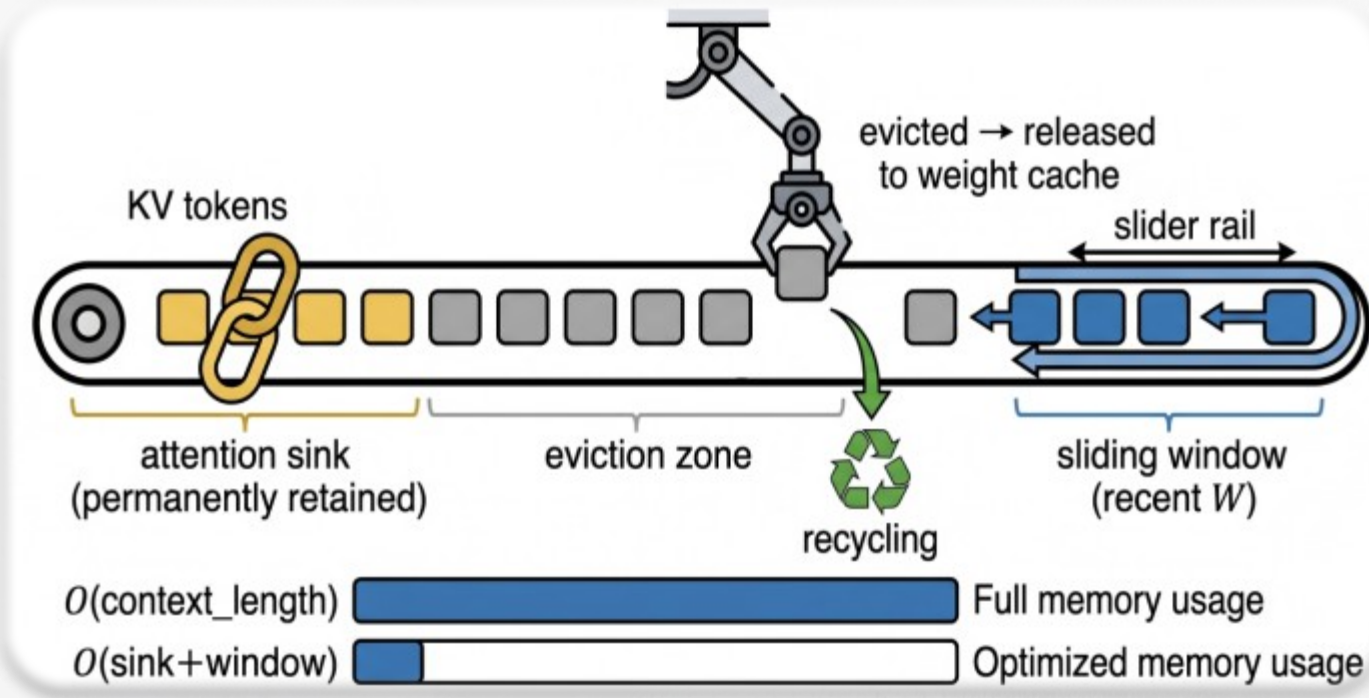
-
- “ ”
- 45GB 8GB OOM
-
- posix_madvise(MADV_RANDOM)
- Page
- Fault





StreamingLLM

-
- KV Cache
- OOM
- “ ”
- **StreamingLLM** 4 Token
- Attention Sink)
-
- Token
- KV



4

SLIM- RC

|

- SLIM_ARC_DISABLE=1 upstream baseline
- SLIM_ARC_NO_MADV_RANDOM=1
MADV_RANDOM
- SLIM_ARC_NO_PREFETCH=1
- SLIM_ARC_DYNAMIC_MADV=1 prefill/decode

| llama.cpp 4

- llama-model-loader.cpp +65 MADV_RANDOM
- llama-context.cpp +85 graph_compute ID
- llama-kv-cache.cpp +7 clear() KV
- CMakeLists.txt +4

| SLIM-ARC

prefetch	
unified-scheduler	I/O
kv-eviction	KV Cache
on-demand	pread


```
1  madv_block = ""mmaps_used.emplace_back(mapping->size(), 0);
2
3      // SLIM-ARC: MADV_RANDOM for large models (>6GB) to enable demand paging.
4      // Tradeoff: prefill slower (no readahead), decode 3-4x faster (MoE sparsity).
5      // SLIM_ARC_NO_MADV_RANDOM=1 to disable.
6      {
7          bool slim_arc_disabled = getenv("SLIM_ARC_DISABLE") != nullptr;
8          bool no_madv = getenv("SLIM_ARC_NO_MADV_RANDOM") != nullptr;
9          size_t msz = mapping->size();
10         if (!slim_arc_disabled && !no_madv && msz > (6ULL << 30) &&
11             mapping->addr() && msz > 0) {
12             (void) posix_madvise(mapping->addr(), msz, POSIX_MADV_RANDOM);
13             // SLIM-ARC: register mmap region for dynamic MADV switching
14             slim_arc::register_mmap_region(mapping->addr(), msz);
15         }
16     }
```



➤ >6GB



cgroup

/sys/fs/cgroup/memory.max

<60%



SLIM_ARC_DISABLE /

SLIM_ARC_NO_MADV_RANDOM

M

◆ MoE Router Hook



```
1 void prefetch_scheduler::worker_loop() {
2     while (true) {
3         int target_layer;
4         uint64_t sig;
5         {
6             std::unique_lock<std::mutex> lk(mtx_);
7             cv_.wait(lk, [this] { return stop_ || target_layer_ != current_layer_.load(); });
8         };
9         if (stop_) return;
10        target_layer = target_layer_;
11        sig = target_signature_;
12    }
13    if (sig != signature_.load()) continue; // stale
14    current_layer_.store(target_layer);
15
16    // Prefetch layers [target+1, target+window]
17    size_t bytes_this_round = 0;
18    for (int w = 1; w <= window_; ++w) {
19        int layer = target_layer + w;
20        if (layer < 0 || (size_t)layer >= tensors_by_layer_.size()) continue;
21        for (const auto & t : tensors_by_layer_[layer]) {
22            if (t.addr == nullptr || t.size == 0) continue;
23            // posix_madvise is thread-safe and idempotent
24            (void) posix_madvise(t.addr, t.size, POSIX_MADV_WILLNEED);
25            bytes_this_round += t.size;
26        }
27    }
28    total_bytes_.fetch_add(bytes_this_round);
29    total_calls_.fetch_add(1);
30 }
31 }
32 }
33 }
```

```
1 if (status == GGML_STATUS_SUCCESS) {
2     if (auto * s = slim_arc::get_global_prefetch_scheduler()) {
3         int n_nodes = ggml_graph_n_nodes(gf);
4         for (int i = 0; i < n_nodes; ++i) {
5             struct ggml_tensor * t = ggml_graph_node(gf, i);
6             if (!t || !t->name) continue;
7             if (strstr(t->name, "ffn_moe_topk") != nullptr && t->data != nullptr) {
8                 int layer = slim_arc::tensor_layer_from_name(t->name);
9                 if (layer < 0) continue;
10                int n_expert_used = (int) t->ne[0];
11                if (n_expert_used <= 0) continue;
12                const int32_t * ed = (const int32_t *) t->data;
13                std::vector<int> ue;
14                for (int e = 0; e < n_expert_used && e < 64; ++e) {
15                    int eid = (int) ed[e];
16                    if (eid >= 0) {
17                        bool found = false;
18                        for (int x : ue) if (x == eid) { found = true; break; }
19                        if (!found) ue.push_back(eid);
20                    }
21                }
22                if (!ue.empty()) s->cache_router_experts(layer, ue.data(), (int) ue.size());
23                break;
24            }
25        }
26    }
27 }
```




```

1 static constexpr double WEIGHT_RATIOS[5][3] =
2 { // {weight, kv, expert}
3   {0.60, 0.10, 0.30}, // PREFILL_SHORT
4   {0.50, 0.20, 0.30}, // PREFILL_LONG
5   {0.70, 0.20, 0.10}, // DECODE_SHORT
6   {0.30, 0.60, 0.10}, // DECODE_LONG
7   {0.20, 0.20, 0.60}, // MOE_DECODE
8 };

```



KV

MoE

I/O



5

× 3

I/O = 15



```

1 if (current_stats_.weight_stalls > 0) {
2   // Weight prefetch is stalling compute → increase weight budget
3   double adjustment = std::min(0.1, current_stats_.weight_stalls * 0.02);
4   budget.weight_bytes = (size_t)(budget.weight_bytes * (1.0 + adjustment));
5   budget.expert_bytes = (size_t)(budget.expert_bytes * (1.0 - adjustment * 0.
6 5)); budget.kv_bytes = (size_t)(budget.kv_bytes * (1.0 - adjustment * 0.5));
7 }

```


StreamingLLM KV Eviction



中山大學
SUN YAT-SEN UNIVERSITY

```
1 kv_evict_block = ""
2 // SLIM-ARC Phase 4: StreamingLLM-style KV eviction after decode.
3 // Keeps first N tokens (attention sinks) + last W tokens (sliding window),
4 // evicts tokens in between via llama_memory_seq_rm to cap KV memory.
5 // Enabled via SLIM_ARC_KV_EVICT=1, configured by SLIM_ARC_KV_SINK/WINDOW.
6 if (status == GGML_STATUS_SUCCESS && !batched) {
7     static bool kv_evict_enabled = getenv("SLIM_ARC_KV_EVICT") != nullptr;
8     if (kv_evict_enabled) {
9         static int kv_sink = []{ const char * e = getenv("SLIM_ARC_KV_SINK"); return e ? atoi(e) : 4; }();
10        static int kv_window = []{ const char * e = getenv("SLIM_ARC_KV_WINDOW"); return e ? atoi(e) : 1024; }();
11        static int last_evicted_pos = 0;
12        static bool first_call = true;
13        if (first_call) {
14            fprintf(stderr, "SLIM-ARC KV eviction: ENABLED (sink=%d, window=%d)\n", kv_sink, kv_window);
15            first_call = false;
16        }
17        llama_pos pos_max = memory->seq_pos_max(0);
18        int seq_len = (int) pos_max + 1;
19        int threshold = kv_sink + kv_window;
20        if (seq_len > threshold) {
21            int p0 = (last_evicted_pos > kv_sink) ? last_evicted_pos : kv_sink;
22            int p1 = seq_len - kv_window;
23            if (p1 > p0) {
24                memory->seq_rm(0, p0, p1);
25                last_evicted_pos = p1;
26                fprintf(stderr, "SLIM-ARC KV eviction: evicted [%d,%d] (seq_len %d->%d)\n",
27                    p0, p1, seq_len, seq_len - (p1 - p0));
28            }
29        }
30    }
31 }
```

➤ sink token

✦

KV

decode ✦9.6%

KV

$O(L)$

$O(1)$

➤ KV Cache clear()

posix_madvise(DONTNEED)

scripts/apply-

slim-arc.py:345-356

5

SLIM- RC

7

CPU	i9-13900H (6P+8E, 20T)
RAM	32 GB DDR5 (WSL2)
	NVMe SSD (~3.5 GB/s)
	AVX2 + AVX_VNNI + FMA
	llama.cpp + SLIM-ARC

8

Qwen3-4B	Q4_K_M	2.4 GB	—
OLMoE-1B-7B	Q4_K_M	3.9 GB	64
Qwen3-Next-80B	IQ4_XS	39.7 GB	512

1



drop_caches

page cache



llama-bench

pp/tg

3



GSM8K 8-shot + WikiText-2

PPL

LEM



upstream llama.cpp

mmap+WILLNEED+KV f16



SLIM- RC

| IQ4_XS

()	pp	tg	
8 GB (4)	—	—	
16 GB (8)	—	2.27 t/s	
32 GB (8)	4.44	3.03 t/s	

| Q4_K_M

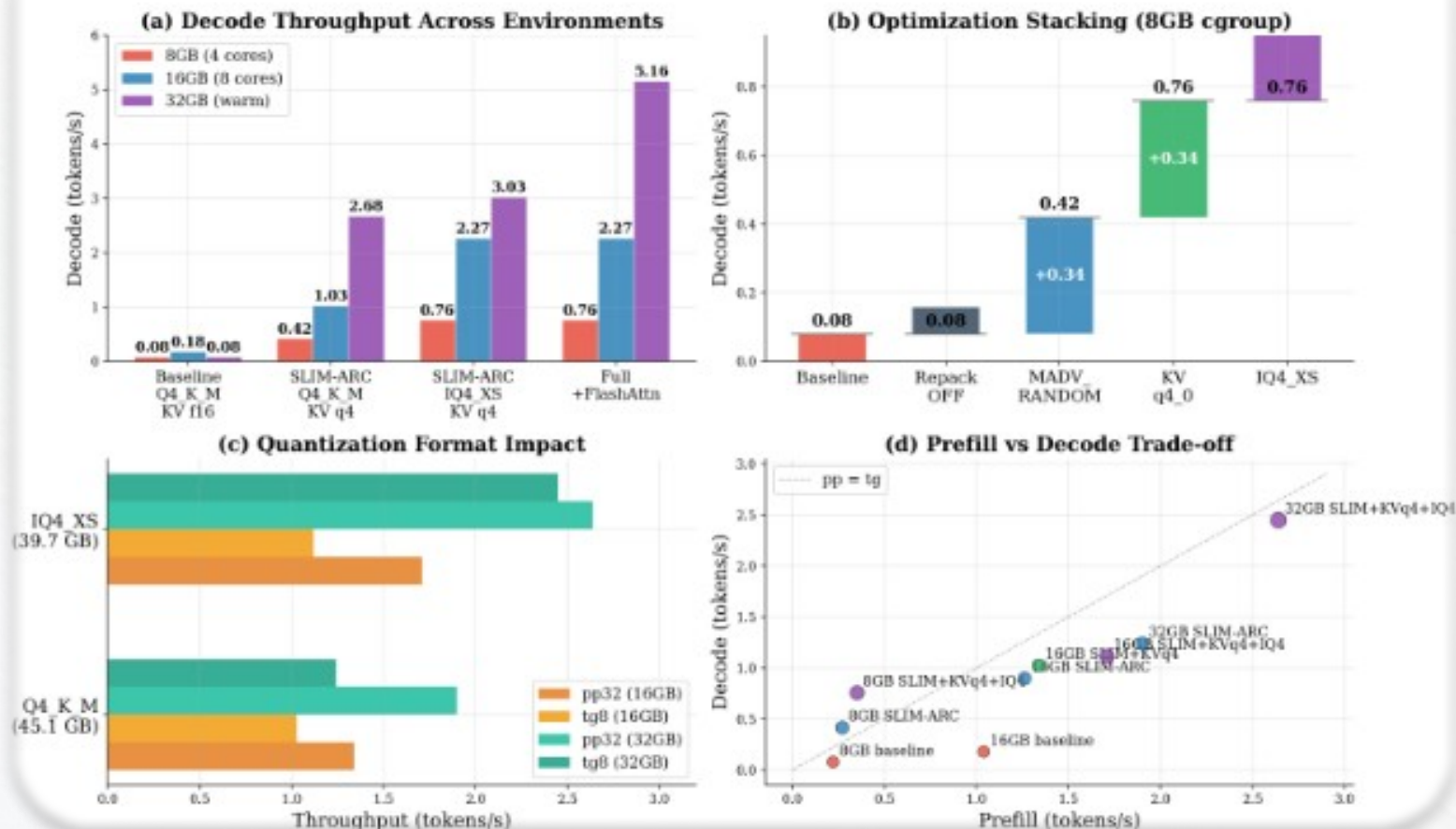
()	pp	tg	
8 GB (4)	—	—	
16 GB (8)	1.96	—	tg
32 GB (8)	—	2.68 t/s	IQ4_XS 12%

- 8GB 40–45GB page fault
0.76 t/s 9.5×
- 16GB 2.27 t/s 80B 6.2×
- 32GB 3.03 t/s IQ4_XS
- Q4_K_M 13%

—— SLIM- RC



SLIM-ARC Performance Landscape: Qwen3-Next-80B



- **64.5×** 8GB baseline 0.08 → 32GB 5.16
- **32GB** Flash attention **+71.4%** 3.01→5.16

8GB→32GB
SLIM-ARC

| 2GB+1

		tg16	
Qwen3-4B	2.4 GB	2.58	
OLMoE-1B-7B	3.9 GB	8.92	

➤ OLMoE MoE 2GB+1
8.92 t/s MADV_RANDOM

➤ IQ4_XS Q4_K_M 0.25 bit
GSM8K 50% 0%
" "

➤ PPL MoE GSM8K

| GSM8K

		GSM8K	t/s
Qwen3-4B	Q4_K_M	75%	7.8
80B	Q4_K_M+KVq4	50%	0.7
80B	IQ4_XS+KVq4	0%	1.7

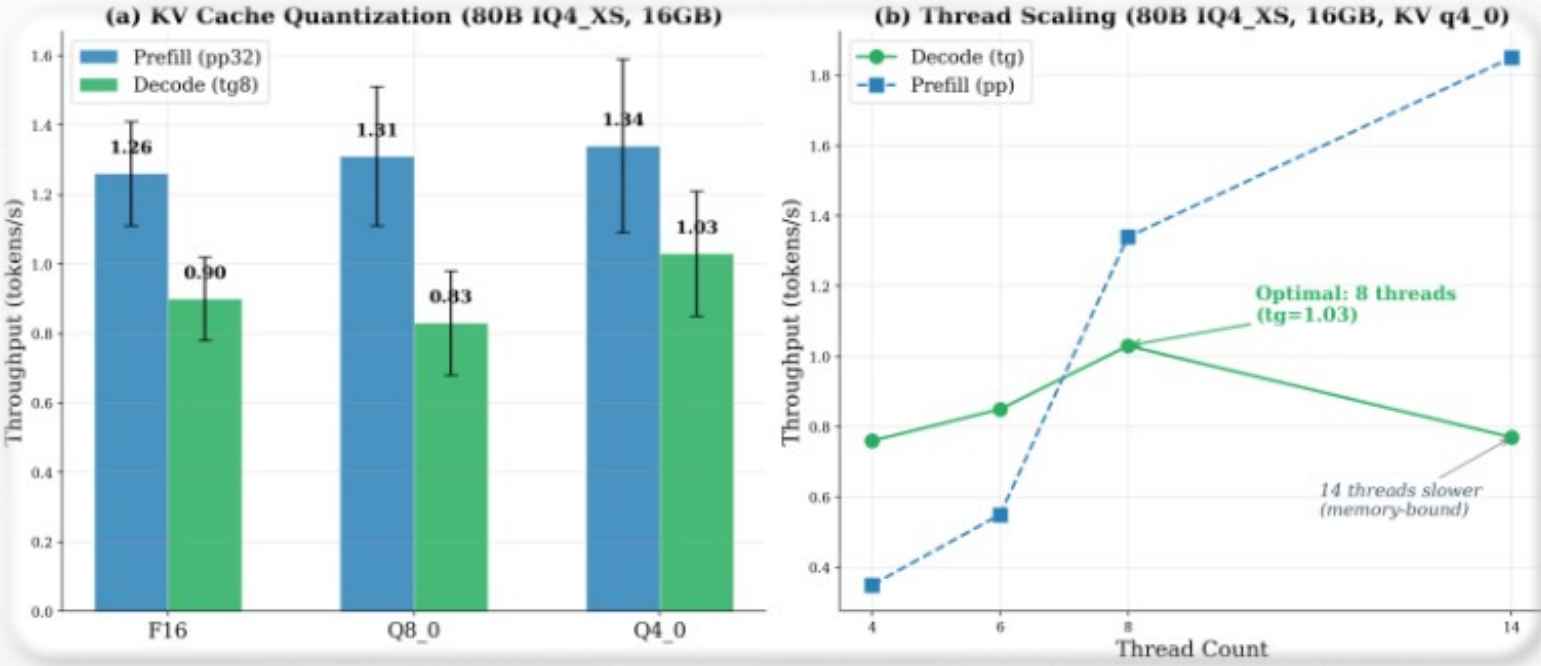
KV Cache

| KV

KV	pp64	tg48	KV	vs Q4_0
F16	—	3.92	2×	↗4%
Q8_0	5.37	2.91	1×	-23%
Q4_0	4.44	3.03	0.5×	best

KV □□□□

- Q4_0 KV 6GB→3GB
- 32GB F16 ↗4%
- 8GB Q4_0 +14%
-



- 8
-
- decode memory-bound NVMe

	pp64	tg48	vs Full	
Full SLIM-ARC	4.44	3.03	—	
- KV q4_0 (f16)	—	3.92	+4%	
- MADV (DISABLE)	3.72	2.15	-41%	
+Eviction	—	3.30	-12%	FA



➤ MADV_RANDOM decode

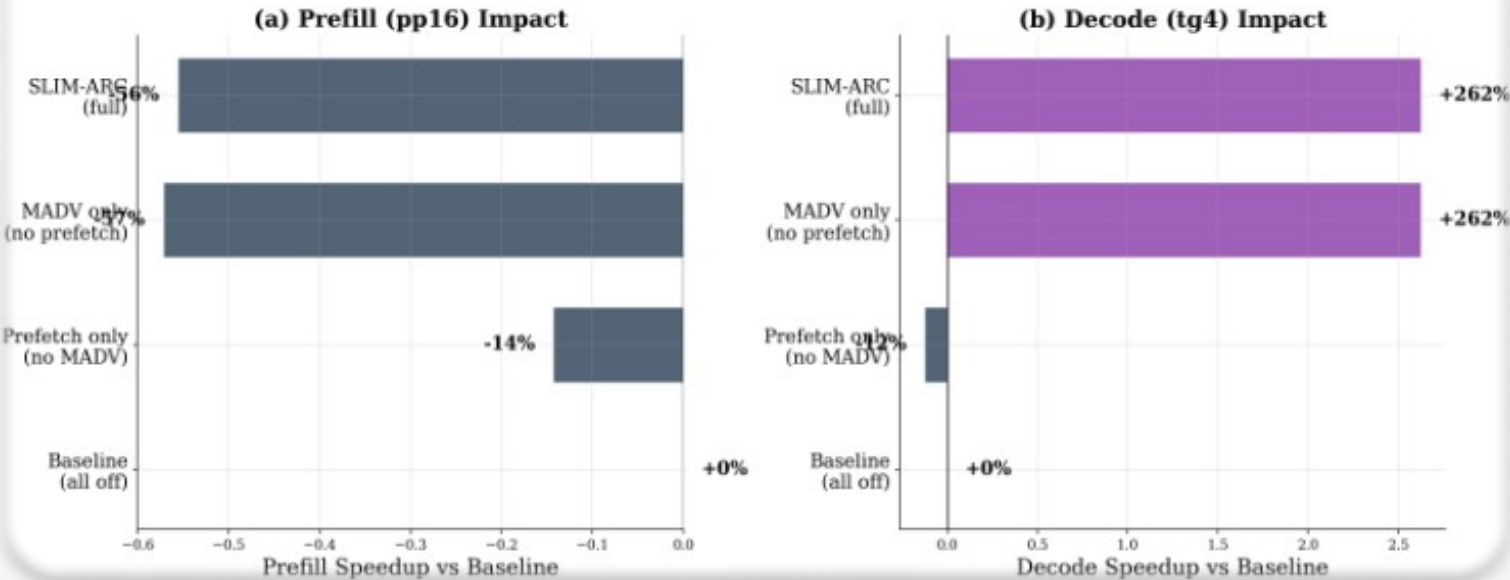
- 41%

➤ Qwen3-Next-80B

token 512 10

2%

Four-Way Ablation: Component Contribution Analysis (80B Q4_K_M, 8GB)

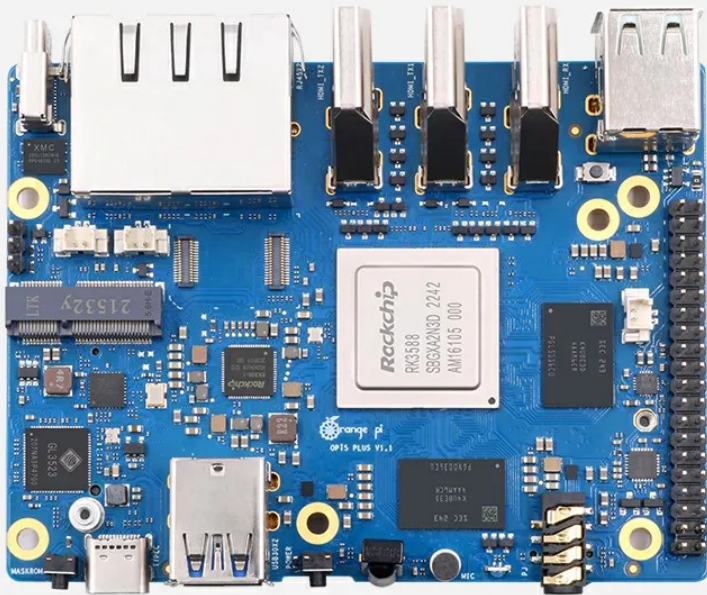


➤ M DV_RANDOM ➡ KV q4_0

➤ Eviction VS Flash tention

Orange Pi 5 Plus

- 瑞芯微RK3588八核 64位处理器
- 主频高达2.4GHz
- 4GB/8GB/16GB (LPDDR4/4x)
- 支持8K视频解码
- 可以外接16GB/32GB/64GB/128GB/256GB eMMC模块
- 2.5G高速双网口
- 支持2230 Wi-Fi 6/BT模块的M.2插槽
- 支持2280 SSD的M.2插槽
- 2个USB 3.0 , 2个USB 2.0



➤	45GB	Qwen3-Next-80B	8GB RK3588
		RSS	OOM
➤		~6.3GB	
		4.5%→34.9%	
		31%→55%	-56%
➤	-c 8192~16384		OOM RSS
	~6.5GB	KV eviction	



6

SLIM- RC

Live Demo



中山大學
SUN YAT-SEN UNIVERSITY

localhost:8090

SLIM-ARC Live Demo
Qwen3-Next-80B-IQ4_XS · 38 GB · 32GB (mmap on-demand)

Qwen3-Next-80B-IQ4_XS (38 GB) 32GB (mmap on-demand)

与 SLIM-ARC 优化后的模型对话，体验端侧 MoE 大模型推理

Assistant

你好！我是 SLIM-ARC 优化后的端侧大模型。在下方输入问题，我会流式回答。右侧面板展示实时系统状态。

User

你是谁

Assistant

我是基于 80B MoE的端侧AI助手，受MADV_RANDOM优化，在8GB内存下运行，能帮你解答问题，无需云端。

User

moe是什么

Assistant

M是指"Mixture of Experts，即"混合专家模型。它通过激活少量参数，实现高效推理，比传统大模型更节省资源，同时保持高性能，是 SLIM-ARC优化

输入问题，回发送...

发送

系统内存

已用 / 总计 5.53 GB / 33.66 GB

Cache (文件缓存) 27.92 GB

推理速度

5.13 t/s

已生成 token 42

MOE 专家激活

10 / 512 激活 稀疏率 98.0%

SLIM-ARC 优化链

A 内核协同 MADV_RANDOM 按需加载

A 内核协同 REPACK=OFF (禁用翻倍)

B 量化压缩 KV q4_0 量化

B 量化压缩 Qwen3-Next-80B-IQ4_XS (38 GB)

C 计算融合 FlashAttention



➤	M DV_R NDOM MoE decode									
	decode		—		tg48	3.03	2.15 t/s	-29%		
➤	GGML_CPU_REP CK									
	repack	-DGGML_CPU_REP CK=OFF			CPU backend			mmap		
➤										
	→ C	'	M DV	B	KV q4_0 / IQ4_XS			'	B	
➤	80B MoE									
	32 GB	IQ4_XS		Qwen3-Next-80B			5.16 tokens/s	decode		
	Flash	ttention								
➤	RK3588 8GB / 5 4GB									
	RK3588	80B	OOM		RSS 6.3GB		31→55%	5		
	Qwen3-4B		OOM		RSS 2.42GiB		.6GB			

LIM-ARC

Email: ouyyp5@mail2.sysu.edu.cn

Website: <https://slim.nexa-lang.com/>

<https://www.bilibili.com/video/BV1fXTF6HEAw/>

	pressure × waste
	model-owned runtime ♦ lease
	schema-1 ♦

64.5×			
850%			
			JSON

-
- /
 - TPS
 - promotion results.json finals-
 - run ID raw
 - manifest
 - cold / warm

generation-token

	prefetched - selected
	expert slice
	POSIX_MADV_DONTNEED

	selected		
/			

- prefetch generation token
- settle / cancel generation
- settle
- flag-off DONTNEED
-

| pressure

	/
normal	85%
high	≥ 85%
critical	≥ 95%
missing	/
	≤90% / ≤75% 2

|

pressure	stable	temporal	hot
normal	*	*	*
high	*		
critical			

| opt-in

- cgroup v2 pressure snapshot
- invalid / overflow missing fallback
- waste EWM ≥60% stable experts
- 2 /
- flag-off

| model-owned runtime ♦ lease

owner	llama_model::impl
activate	mmap
acquire	move-only runtime lease
deactivate	→→→ join
destroy	runtime model mapping

|

2 GiB	pp64/tg16	exact 2	base=0
4 CPU	same seed	cold 1	patch=1
swap 0	5 configs	warm 1	schema -1

|

- (generation, layer)
- scheduler mutex madvise
-
-
- cold / warm